

Chapter 1: Introduction

1.1 Abstract

Agriculture remains the backbone of economies worldwide, particularly in developing nations where crop production directly determines food security and farmer livelihoods. Despite its importance, agricultural yield forecasting has traditionally depended on expert agronomists conducting field surveys, historical yield averages, and simplistic linear models that fail to capture the complex interactions among climate, soil, and farming practice variables. Machine learning offers a transformative alternative: by learning patterns from large historical datasets, ML models can deliver accurate, near-instantaneous yield predictions without requiring field visits or domain expertise.

This project presents the Crop Yield Prediction System, a Python desktop application that uses a Random Forest Regressor to predict the yield of a specific crop in a specific country, expressed in hectograms per hectare (hg/ha), based on six input features: country (Area), crop type (Item), harvest year, average annual rainfall, pesticide usage, and average temperature. The system is trained on the Kaggle Crop Yield Prediction Dataset comprising 28,242 records spanning 101 countries, 10 major crops, and the years 1990–2013.

The application delivers a complete ML pipeline within a single Python file using only five standard libraries: pandas, NumPy, scikit-learn, matplotlib, and Joblib. Categorical features (Area and Item) are label-encoded before training. A Random Forest Regressor with 200 trees achieves R^2 of 0.93+ on the held-out test set. The Tkinter GUI provides dropdown-based selection of country and crop from the training data vocabulary, numeric entry for climate inputs, and a colour-coded yield rating (Very High / High / Moderate / Low). Four Joblib artefacts (model, scaler, area encoder, item encoder) are saved on demand and loadable in future sessions.

1.2 Introduction

Global food demand is projected to increase by 50% by 2050 as population grows and dietary patterns shift toward more resource-intensive foods. Meeting this demand requires optimizing agricultural productivity through better forecasting, resource allocation, and policy planning. Crop yield — the amount of agricultural product harvested per unit of land — is the central quantity that determines

whether a harvest will be sufficient to feed a population, turn a profit for a farmer, or meet export commitments.

Yield is determined by a complex interaction of agronomic factors (crop variety, planting density, irrigation), environmental factors (rainfall, temperature, sunlight), and input factors (fertilizers, pesticides, mechanization). Traditional statistical models, such as linear regression of yield on rainfall and temperature, fail to capture the non-linear interactions and categorical structure of this data. Random Forest Regression — an ensemble of decision trees that votes on a continuous output — is well suited to this problem: it handles mixed feature types (categorical country/crop codes alongside continuous climate values), captures non-linear interactions, and provides feature importance rankings that help agronomists understand which variables drive predictions.

This project brings these capabilities to a user-friendly desktop application accessible to students, researchers, and agricultural extension workers without requiring programming knowledge. The combination of dropdown menus (populated automatically from the training data), numeric entry fields, and visual metric output makes the application intuitive and immediately usable after training.

1.3 Advantages

- Predicts crop yield for 10 crops across 101 countries from a single trained model.
- Handles categorical features (country, crop) using LabelEncoder — no one-hot encoding overhead.
- Dropdown menus are populated automatically from the training data, preventing invalid entries.
- Saves four Joblib artefacts (model, scaler, two encoders) enabling fully offline repeated use.
- Random Forest achieves $R^2 > 0.93$ — captures complex climate-crop interactions that linear models miss.
- Colour-coded yield rating (Very High/High/Moderate/Low) provides instant actionable feedback.
- Actual vs Predicted scatter plot gives immediate visual confirmation of model quality.
- Feature importance chart reveals which agronomic inputs most influence predicted yield.
- No internet connection required after dataset download and initial setup.
- Single-file architecture with no database or server dependency — runs on any Python 3.8+ machine.

1.4 Applications

- National agricultural ministries forecasting harvest volumes for food security planning.
- Farmers selecting optimal crop types for their region, year, and climate conditions.
- Insurance companies estimating expected yield to price crop insurance premiums.
- NGOs identifying low-yield regions requiring agronomic intervention or aid.
- Agricultural researchers benchmarking new regression algorithms on a multi-country dataset.
- University courses demonstrating label encoding, regression evaluation, and feature importance.
- Export planners estimating available surplus for international commodity trading.
- Climate adaptation programs modelling yield impact of projected temperature and rainfall changes.

1.5 Existing System

Current crop yield estimation methods include crop simulation models (DSSAT, APSIM) which simulate plant physiology day-by-day but require detailed soil, weather, and management inputs not available at national scale; remote sensing approaches (MODIS, Sentinel satellite data) which estimate yield from vegetation indices but require image processing expertise and cloud-free imagery; and statistical regression tools embedded in agricultural software packages (e.g., SAS Enterprise, SPSS) which are commercial, expensive, and do not provide interactive prediction interfaces. Government agencies such as the FAO and USDA publish historical yield data but provide no prediction capability. No freely available, open-source desktop application currently integrates multi-country multi-crop yield prediction with an interactive GUI, label-encoded categorical inputs, and automated feature importance visualization in a single Python file deployable without server infrastructure.

1.6 Disadvantages of Existing Systems

- Crop simulation models (DSSAT, APSIM) require soil data and daily weather records not always available.
- Remote sensing tools require image processing expertise and expensive cloud subscriptions.

- Commercial statistical packages (SAS, SPSS) carry high licensing costs.
- FAO and USDA tools provide historical lookup only — no predictive inference for new inputs.
- Existing ML notebooks lack a packaged, interactive desktop application interface.
- No existing tool provides automatic dropdown population from training data vocabulary.
- Most solutions lack feature importance visualization for agronomic insight.
- Web-based tools require internet access and raise data sovereignty concerns for national agencies.

1.7 Objectives

1. Design and implement a Python desktop application to predict crop yield (hg/ha) using Random Forest Regression.
2. Support multi-country, multi-crop datasets with automatic label encoding of categorical features.
3. Populate prediction dropdowns automatically from the vocabulary of the training dataset.
4. Evaluate the model using MAE, RMSE, and R^2 Score on a stratified 80/20 train-test split.
5. Visualize model diagnostics: Actual vs Predicted scatter plot and feature importance bar chart.
6. Persist model, scaler, and both LabelEncoders as four Joblib .pkl files for offline reuse.
7. Provide a colour-coded yield quality rating (Very High/High/Moderate/Low) alongside the numeric prediction.
8. Deliver the complete system as a role-authenticated, single-file Tkinter desktop application.

1.8 Scope

The Crop Yield Prediction System is scoped to the 10 crop types and 101 countries present in the Kaggle training dataset, covering harvest years 1990–2013. Predictions for crops, countries, or climate conditions outside this training distribution may be less reliable. The system performs batch dataset training from CSV and individual sample prediction via form input. It does not support real-time sensor data ingestion, multi-year time-series forecasting, or sub-national (district/province) level predictions. These are identified as future extensions in Chapter 6.

Chapter 2: System Analysis

2.1 Introduction

System analysis for the Crop Yield Prediction System addressed the unique challenges of agricultural ML: categorical features representing geographic regions and crop types, a large dataset with records spanning over two decades, and the need to save not just the trained model and scaler but also the label encoders that map string values to integers. The analysis confirmed that all requirements are achievable within the BCA project scope using freely available tools.

2.2 Feasibility Study

2.2.1 Operational Feasibility

The application is designed for agricultural researchers, students, and extension workers with basic computer literacy. The use of dropdown menus for country and crop selection — populated directly from the training data — eliminates the possibility of invalid categorical inputs and simplifies the user experience significantly compared to free-text entry. The four-tier yield rating system (Very High to Low) translates the raw hg/ha number into an immediately understandable quality indicator. The role-based login provides basic access control appropriate for shared laboratory computers.

2.2.2 Technical Feasibility

The Kaggle dataset (28,242 rows, 7 columns) is small enough to load into RAM in under one second and train a 200-tree Random Forest in approximately 10–15 seconds on a standard personal computer. Label encoding with scikit-learn's LabelEncoder is deterministic — the same string always maps to the same integer given the same fitted encoder — making it safe to save and reload the encoder for use in prediction sessions independent of the training session. All required libraries are freely available via pip.

2.2.3 Economic Feasibility

Total development cost is zero in terms of software licenses. Python, all five libraries, and the Kaggle dataset are freely available. The Kaggle dataset requires free registration to download. No cloud infrastructure, database server, or paid API is required. Hardware requirements are modest — any computer with 4 GB RAM running Python 3.8+ is sufficient.

2.3 Software Requirements

Component	Requirement	Version / Notes
Operating System	Windows 10/11, macOS 12+, Ubuntu 20.04+	Any 64-bit OS
Programming Language	Python	3.8 or higher
GUI Framework	Tkinter + ttk	Bundled with Python
Data Processing	pandas	>= 1.5.0
Numerical Computing	NumPy	>= 1.23.0
Machine Learning	scikit-learn	>= 1.2.0
Visualization	matplotlib	>= 3.6.0
Model Persistence	Joblib	>= 1.2.0
Development IDE	VS Code / PyCharm Community	Optional
Dataset Source	Kaggle Crop Yield Prediction Dataset	Free — Kaggle account required

2.4 Tools and Technologies

Tool / Technology	Category	Purpose in This Project
Python 3.x	Language	Core application logic and ML pipeline
Tkinter + ttk	GUI Framework	Windows, tabs, dropdowns, entry fields, event handling
pandas	Data Library	CSV loading, column normalisation, missing-value handling

Tool / Technology	Category	Purpose in This Project
NumPy	Numerical Library	Array construction, RMSE calculation via numpy.sqrt
scikit-learn	ML Library	RandomForestRegressor, LabelEncoder, StandardScaler, metrics
matplotlib	Visualization	Actual vs Predicted scatter, feature importance bar chart
Joblib	Serialization	Save/load model, scaler, area encoder, item encoder (4 files)
Kaggle Crop Yield Dataset	Dataset	28,242 records, 10 crops, 101 countries, hg/ha target

2.5 Module Specification

Module No.	Module Name	Description
M01	Authentication Module	Validates username/password; routes Admin/User to dashboard; Enter-key binding on login form.
M02	Dataset Upload Module	Opens file dialog for CSV files; normalises column names; applies rename_map for alternate column-name variants; validates all required columns exist; converts numeric columns; drops NaN rows; extracts sorted unique lists of countries and crops for dropdown population.
M03	Label Encoding Module	Fits separate LabelEncoder instances on the Area (country) and Item (crop) columns of the training DataFrame; transforms both columns to integer codes; stores encoders in g_enc_area and g_enc_item global variables.
M04	Model Training Module	Performs 80/20 random train/test split; fits StandardScaler on training features; trains RandomForestRegressor (n_estimators=200, max_depth=15, n_jobs=-1) on scaled training data; generates predictions on the scaled test set.
M05	Evaluation Module	Computes MAE, RMSE (numpy.sqrt of MSE), and R ² score; updates three metric cards; re-renders two matplotlib charts on every training run.

Module No.	Module Name	Description
M06	Visualization Module	Chart 1: Actual vs Predicted scatter plot of a 500-sample subset (for rendering speed), with red dashed perfect-fit reference line and axes in $\times 1000$ hg/ha units. Chart 2: Feature importance horizontal bar chart with human-readable feature labels, sorted ascending. Both charts embedded via FigureCanvasTkAgg.
M07	Model Save Module	Calls joblib.dump() for four artefacts: model, scaler, area encoder, item encoder. All written to the /model directory.
M08	Model Load Module	Four sequential file dialogs (model, scaler, area encoder, item encoder); calls joblib.load(); restores dropdown values from loaded encoder classes.
M09	Prediction Module	Reads dropdown selections (country, crop); reads four numeric Entry fields; label-encodes string selections using loaded encoders; constructs NumPy array; applies scaler.transform(); calls model.predict(); maps numeric hg/ha to yield rating with colour badge.
M10	Status and Navigation Module	Persistent green-tinted status bar updated after every operation; NotebookTabChanged event refreshes dropdown values when user switches to Predict tab.

Chapter 3: System Methodology

3.1 Architecture Overview

The Crop Yield Prediction System extends the single-file layered architecture used in Projects 1 and 2 with one significant addition: the Label Encoding Layer. Unlike the previous two projects, which used only continuous features, this project includes two categorical string features (country name and crop type) that must be converted to integers before they can be fed to a scikit-learn estimator. This introduces two new global state variables (`g_enc_area`, `g_enc_item`) and two new model artefacts that must be saved and loaded alongside the model and scaler.

The architecture therefore has four persistent artefacts instead of two: the trained `RandomForestRegressor`, the fitted `StandardScaler`, the fitted `LabelEncoder` for Area, and the fitted `LabelEncoder` for Item. Separating the encoders from the scaler and model is intentional: it allows any component to be updated independently if the dataset vocabulary grows (new countries or crops added) without retraining the full model.

3.2 Application Design

3.2.1 Categorical Feature Handling

The Crop Yield dataset contains two string columns — Area (country name) and Item (crop type) — that are nominal categorical variables with no natural ordering. Label encoding assigns a unique integer to each distinct string value (e.g., Albania=0, Algeria=1, ..., Zimbabwe=100 for Area; Cassava=0, Maize=1, ..., Yams=9 for Item). While label encoding implies an arbitrary ordinal relationship that tree-based models do not rely on for split decisions, it is the most memory-efficient encoding for high-cardinality categoricals and avoids the dimensionality explosion of one-hot encoding (which would add 100 binary columns for Area alone).

The `LabelEncoder` is fitted on the full dataset (not just the training split) to ensure it covers all known country and crop values. This means the encoder vocabulary encompasses all entries the user might select from the dropdown, eliminating unknown-label errors during prediction. The encoder's `classes_` attribute is used to populate the `ttk.Combobox` widget values.

3.2.2 Dropdown Population Strategy

A key UX design decision is the automatic population of the Country and Crop dropdowns from the training data vocabulary. This is implemented using a NotebookTabChanged event binding: whenever the user switches to the Predict tab, the refresh_dropdowns() callback checks whether g_area_list and g_item_list are populated and updates the Combobox values accordingly. This lazy-refresh approach means the dropdowns are always current even if the user trains multiple times with different datasets in the same session. Both Comboboxes are set to state='readonly', preventing free-text entry that could produce unknown-label errors.

3.2.3 Yield Rating System

The yield_rating() function maps the raw hg/ha prediction to a four-tier quality label. The thresholds (100,000 / 50,000 / 20,000 hg/ha) are derived from the dataset distribution: the top quartile of yield values in the training data exceeds 100,000 hg/ha (crops like sugarcane and potatoes with extremely high volumetric yields), while the bottom quartile falls below 20,000 hg/ha (dryland cereals in arid regions). Each tier carries a distinct foreground and background colour applied to the rating label widget, providing immediate visual differentiation.

3.3 Data Schema

The Kaggle Crop Yield Prediction Dataset schema after loading and preprocessing:

Column Name	Data Type	Description	Example Values
Area	String → Int	Country / region name (label-encoded for training)	Albania, Brazil, India
Item	String → Int	Crop type (label-encoded for training)	Maize, Rice, Wheat, Potatoes
Year	Integer	Harvest year	1990 – 2013
average_rain_fall_mm_per_year	Float	Mean annual precipitation in mm	51 – 3,240
pesticides_tonnes	Float	Total pesticides applied in metric tonnes	0.01 – 367,778

Column Name	Data Type	Description	Example Values
avg_temp	Float	Mean annual temperature in degrees Celsius	1.3 – 30.0
hg/ha_yield	Float	TARGET: crop yield in hectograms per hectare	845 – 501,412

Four Joblib artefacts saved by the application:

- model/crop_yield_model.pkl — serialized RandomForestRegressor (200 trees, max_depth=15).
- model/crop_yield_scaler.pkl — serialized StandardScaler fitted on training feature matrix.
- model/encoder_area.pkl — serialized LabelEncoder for country names (101 classes).
- model/encoder_item.pkl — serialized LabelEncoder for crop types (10 classes).

3.4 Software Engineering Approach

The project uses the same iterative prototyping approach as Projects 1 and 2, with four development passes. Pass 1: login and tabbed dashboard skeleton. Pass 2: dataset loader with rename_map for column normalisation and Treeview preview. Pass 3: label encoding, training pipeline, metric cards, and chart rendering. Pass 4: dropdown-based prediction form, yield rating badge, and four-artefact save/load.

A notable engineering decision was to bind dropdown refresh to the NotebookTabChanged event rather than to the training completion callback. This decoupling means the dropdowns are refreshed every time the Predict tab is visited, making the behaviour robust to training being triggered multiple times or model loading being done out of order. The alternative — refreshing dropdowns inside the train() function — would fail to update the dropdowns if the user loaded a model (which does not call train()) rather than training fresh.

3.5 Technology Deep-Dives

3.5.1 Python and Global State

Six global variables manage shared application state: `g_model` (RandomForestRegressor), `g_scaler` (StandardScaler), `g_enc_area` (LabelEncoder for country), `g_enc_item` (LabelEncoder for crop), `g_df` (pandas DataFrame), `g_area_list` (sorted list of country strings), and `g_item_list` (sorted list of crop strings). The list variables are required separately from the encoders because the encoder classes' `classes_` attribute returns a NumPy array, while `ttk.Combobox` values expects a Python list.

3.5.2 Tkinter ttk.Combobox

The `ttk.Combobox` widget is used for Country and Crop selection. It is initialised with an empty values list and updated via the `refresh_dropdowns()` callback. The `state='readonly'` parameter prevents keyboard entry, ensuring only valid training-data vocabulary values can be selected. The `textvariable` parameter links the combobox to a `tk.StringVar` (`cb_area_var`, `cb_item_var`), allowing the selected value to be read in the `predict()` function without querying the widget directly. This separation of widget state from logic follows standard Tkinter best practices.

3.5.3 Random Forest Regressor

The RandomForestRegressor with `n_estimators=200` and `max_depth=15` is configured to balance bias and variance on the 28,242-row dataset. `max_depth=15` allows trees to capture complex interactions (e.g., the interaction between country, crop, and temperature) while preventing overfitting that would occur with fully-grown trees (`max_depth=None`). `n_jobs=-1` uses all available CPU cores for parallel tree construction, reducing training time from ~30 seconds to ~10 seconds on a quad-core laptop. `random_state=42` ensures reproducible results across runs.

The Random Forest handles the label-encoded categorical features correctly because tree splits are based on inequality thresholds (e.g., `Item_encoded <= 4`), which effectively partition the encoded integer space into subsets regardless of the arbitrary ordering. The model learns that, for example, splits on `Item_encoded` isolate wheat records from potato records not because `wheat < potato` in any meaningful sense but because one integer range correlates with high yield and another with low yield in the training data.

3.5.4 LabelEncoder and Prediction Safety

During prediction, `g_enc_area.transform([area_str])` converts the user's selected country string to its integer code. If the user somehow selects a value not in the encoder's vocabulary (e.g., by directly

editing the Combobox despite readonly state, or by loading a model trained on a different dataset), a `ValueError` is raised and caught by the except block, which displays an informative error dialog. This defensive pattern prevents silent incorrect integer assignments that would produce nonsensical predictions.

3.5.5 Joblib Multi-Artefact Save/Load

Saving four artefacts requires four `joblib.dump()` calls. Loading requires four sequential file dialogs — one for each artefact — which is the most transparent approach for students: they explicitly select each file and can verify they are loading the correct artefacts for a given training session. An alternative design (saving all four artefacts into a single dictionary and dumping that) would require only one file dialog but would increase coupling between the artefacts. The multi-file approach is preferred here for educational clarity and independent replaceability.

3.5.6 Chart Optimisation for Large Datasets

The Actual vs Predicted scatter plot uses a 500-sample random subset of the test set rather than plotting all test points. With 28,242 training rows and an 80/20 split, the test set contains ~5,650 rows — plotting all as scatter points produces an over-plotted chart where individual points are invisible. The 500-sample subset preserves the visual distribution of predictions while rendering in under one second. The axis units are divided by 1,000 (yielding $\times 1000$ hg/ha) to keep tick labels readable. Feature importance values are displayed for six features with human-readable labels substituted for internal column names (e.g., 'Rainfall' instead of 'average_rain_fall_mm_per_year').

Chapter 4: User Interface

4.1 Overview

The Crop Yield Prediction System uses a green-accented colour theme (primary colour #1D6B3E — forest green) to reflect the agricultural domain, distinguishing it visually from the blue-themed Projects 1 and 2. The same structural layout is retained: Login window, three-tab Dashboard (Dataset, Train, Predict), header bar, and status bar. The status bar uses a light-green background (#E4F0E8) with dark-green text to maintain theme consistency. The key GUI innovation in this project is the use of `ttk.Combobox` widgets for categorical inputs, replacing the free-text `Entry` widgets used in previous projects.

4.2 Login Screen

The Login Screen is a fixed 430 x 410 pixel window presenting a wheat emoji icon, the application title in forest green, and a subtitle identifying the algorithm. The login mechanics are identical to Projects 1 and 2: username/password `Entry` fields, a Login button, an Enter-key binding, and a red error message label for invalid credentials. The two credential pairs (admin/admin123, user/user123) are displayed in small grey text at the bottom of the window.

4.3 Dashboard — Dataset Tab

The Dataset Tab header displays an information label that, after loading, reports the row count, number of unique crops, number of unique countries, and the minimum and maximum yield values in hg/ha. The Upload button opens a file dialog filtered to `.csv` files. After loading, the Treeview widget displays the first 100 rows of the preprocessed `DataFrame` with seven columns (Area, Item, Year, average_rain_fall_mm_per_year, pesticides_tonnes, avg_temp, hg/ha_yield). Column widths are set to 120 pixels with a minimum of 80 pixels. Float values are shown to two decimal places.

4.4 Dashboard — Train Model Tab

The Train Model Tab presents three metric cards: MAE (blue), RMSE (blue), and R^2 Score (green). MAE and RMSE are formatted with thousand-separator commas (e.g., '12,450') to keep large hg/ha numbers readable. Below the cards, the chart area renders two panels: the Actual vs Predicted scatter plot in forest green with a red dashed perfect-fit line (left), and the feature importance bar chart in teal (right). The three action buttons — Train Model (green), Save Model (blue), Load Model (amber) — are arranged horizontally below the charts. The Load Model button triggers four sequential file dialogs, guiding the user through selecting the model, scaler, area encoder, and item encoder files in order.

4.5 Dashboard — Predict Yield Tab

The Predict Yield Tab uses a 3-column grid layout with six input widgets. The first row contains three widgets: a Combobox for Country selection (Col 0), a Combobox for Crop selection (Col 1), and a Year Entry field (Col 2). The second row contains three numeric Entry fields for Rainfall (Col 0), Pesticides (Col 1), and Average Temperature (Col 2). All Entry fields are pre-populated with representative median defaults. Comboboxes are populated when the user switches to the tab (lazy refresh via NotebookTabChanged event) and show a sorted alphabetical list of all countries and crops from the training data.

The result area below the inputs contains three stacked elements: a large bold yield value label (e.g., 'Predicted Yield: 87,432 hg/ha') coloured in the rating's foreground colour; a colour-coded rating badge label (e.g., 'Yield Rating: High' with blue background); and a subtitle line showing the selected crop, country, and year. The Predict Yield (green) and Clear (grey) buttons control prediction and field reset. If no model is loaded, a warning dialog prompts training or loading.

Chapter 5: Testing

5.1 Introduction to Testing

Testing of the Crop Yield Prediction System focused on three unique challenges not present in the previous two projects: (1) label encoding correctness — verifying that string-to-integer transformation is consistent between training and prediction; (2) dropdown population — confirming that Combobox values update correctly after training and after loading a saved model; and (3) four-artefact save/load — verifying that all four .pkl files are correctly written and restored. All tests were performed manually on Windows 11 running Python 3.11 with scikit-learn 1.3.0.

5.2 Unit Testing

Key units tested: the `rename_map` column normalisation (verified that datasets with alternate column names such as 'rainfall_mm' and 'temperature' are correctly mapped); the label encoding round-trip (encoded a country string, decoded it back, verified it matched the original); the `yield_rating()` function at boundary values (19,999; 20,000; 49,999; 50,000; 99,999; 100,000 hg/ha); the `refresh_dropdowns()` callback (verified Combobox values update when switching to the Predict tab after training); and the four-artefact save (verified all four .pkl files are created in the /model directory with non-zero file sizes).

5.3 Integration Testing

Integration testing traced two end-to-end paths. Path 1 (train and predict): upload CSV → train → save four artefacts → switch to Predict tab → verify dropdowns populated → select India / Wheat / 2010 / 1100mm / 45000t / 25°C → predict → verify result is a plausible hg/ha value with a rating badge. Path 2 (load and predict): fresh application launch → load four artefacts → switch to Predict tab → verify dropdowns populated from loaded encoder classes_ → predict same inputs as Path 1 → verify result matches Path 1 within floating-point tolerance.

5.4 Test Cases

TC#	Test Case Description	Input / Action	Expected Output	Result
TC01	Valid Admin Login	admin / admin123	Dashboard opens, header shows ADMIN	PASS
TC02	Valid User Login	user / user123	Dashboard opens, header shows USER	PASS
TC03	Invalid Login Credentials	admin / wrong	Red error message displayed	PASS
TC04	Upload Valid Crop Yield CSV	crop_yield.csv selected	28,242 rows, 10 crops, 101 countries reported	PASS
TC05	Column Rename Map Test	CSV with 'rainfall_mm' column	Loaded correctly — renamed to standard column name	PASS
TC06	Train Model on Dataset	Click Train Model	MAE, RMSE, R ² update; scatter + importance charts render	PASS
TC07	Train Without Dataset	Click Train before upload	Warning: 'Upload a dataset first'	PASS
TC08	R ² Score Range Check	Train on standard dataset	R ² value between 0.90 and 0.97	PASS
TC09	Save Model (4 files)	Click Save Model after training	Four .pkl files created in /model/ directory	PASS
TC10	Load Model (4 dialogs)	Select each of 4 .pkl files	Success dialog; dropdowns populated from encoder classes_	PASS
TC11	Dropdown Population After Train	Switch to Predict tab after training	Comboboxes show all 101 countries and 10 crops	PASS
TC12	Predict with Default Values	Click Predict (defaults pre-filled)	Yield value displayed with colour-coded rating badge	PASS

TC#	Test Case Description	Input / Action	Expected Output	Result
TC13	Predict Very High Yield Crop	Select Potatoes, irrigated region	Green badge: 'Very High'	PASS
TC14	Non-Numeric Input in Year Field	Enter text in Year field	Error dialog: 'Year, Rainfall... must be numeric'	PASS
TC15	Clear Predict Fields	Click Clear button	Entry fields reset to defaults; result labels cleared	PASS

5.5 System Testing Results

All 15 test cases passed on first execution. The Random Forest Regressor trained on 28,242 rows achieved R^2 of 0.93–0.95 on the 20% held-out test set, with MAE of approximately 8,000–12,000 hg/ha and RMSE of 15,000–20,000 hg/ha. Training time was approximately 12–18 seconds on an Intel Core i5 quad-core laptop using `n_jobs=-1` (all cores). The 500-point scatter plot rendered in under 0.5 seconds. All four .pkl artefacts were saved and reloaded successfully, with the reloaded model producing identical predictions to the in-session model. The NotebookTabChanged-based dropdown refresh worked correctly in both the post-training and post-load scenarios.

Chapter 6: Conclusion

6.1 Summary of Achievements

The Crop Yield Prediction System successfully delivers a regression ML pipeline for agricultural yield forecasting as a standalone desktop application. The following goals were achieved:

- A Random Forest Regressor trained on 28,242 crop records achieves R^2 of 0.93–0.95, demonstrating excellent predictive power across 10 crops and 101 countries.
- Label encoding of categorical country and crop features is handled transparently within the application, with encoders saved as separate Joblib artefacts and used for prediction input transformation.
- Dropdown menus populated automatically from training data vocabulary eliminate the possibility of invalid categorical inputs and improve usability for non-technical users.
- A four-tier colour-coded yield rating system (Very High/High/Moderate/Low) translates raw hg/ha predictions into immediately actionable quality indicators.
- Three evaluation metrics (MAE, RMSE, R^2) and two diagnostic visualizations (scatter plot, feature importance) provide a comprehensive view of model performance.
- Four Joblib artefacts (model, scaler, area encoder, item encoder) enable fully offline repeated predictions without retraining.

6.2 Technical Reflections

This project introduced label encoding as a categorical feature handling strategy and highlighted its trade-offs. Label encoding is fast and memory-efficient but introduces an implicit ordinal relationship between category codes that tree-based models handle correctly (by learning splits based on code ranges) but linear models would misinterpret (assuming Albania < Algeria < ... in a meaningful numeric sense). Understanding this algorithm-dependency of encoding choice is a core lesson in applied ML.

The four-artefact save/load pattern — compared to the two-artefact pattern of Projects 1 and 2 — illustrates how the complexity of model persistence grows with the preprocessing pipeline. In production systems, it is common practice to wrap the entire pipeline (encoders, scaler, model) into a single scikit-learn Pipeline object and serialize the Pipeline as one artefact. This project deliberately

uses separate artefacts for pedagogical transparency: students can inspect and understand each component independently.

The chart optimisation (500-sample subset for scatter plot) addresses a practical performance issue that students encounter when scaling from toy datasets to real-world data. Plotting 5,650 overlapping points is both slow and visually uninformative. Random subsampling preserves the statistical distribution of predictions while dramatically improving rendering speed and chart readability. This technique generalises to any visualization task involving dense point clouds.

6.3 Future Scope

- **Pipeline Serialization:** Wrap the full preprocessing and modelling pipeline into a scikit-learn Pipeline object for single-artefact save/load.
- **Weather API Integration:** Connect to the Open-Meteo or WeatherAPI to auto-fill rainfall and temperature fields based on the selected country and year.
- **Seasonal Granularity:** Extend the feature set to include planting season (Kharif/Rabi/Zaid) and soil type for finer-grained regional predictions.
- **Multi-Year Forecasting:** Add a time-series forecasting module using ARIMA or LSTM to project yield trends 3–5 years ahead.
- **Geospatial Map Visualization:** Integrate folium or geopandas to display predicted yield on a world map with country-level colour shading.
- **Web Deployment:** Port to Streamlit for mobile-browser access by farmers in the field.
- **PDF Agronomic Report:** Export a formatted PDF containing prediction, input parameters, yield rating, feature importance chart, and agronomic recommendations using the reportlab library.

In conclusion, the Crop Yield Prediction System demonstrates that machine learning can make a meaningful contribution to agricultural decision-making through an accessible, interpretable, and offline-capable desktop application. The project advances beyond Projects 1 and 2 by introducing categorical encoding, multi-artefact persistence, and domain-specific output formatting, providing BCA students with a progressively richer ML engineering experience.

Chapter 7: Bibliography

Books

- Geron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
- McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.
- Raschka, S., & Mirjalili, V. (2019). Python Machine Learning (3rd ed.). Packt Publishing.
- Brownlee, J. (2020). Machine Learning Mastery with Python. Machine Learning Mastery.
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.

Websites

- Kaggle — Crop Yield Prediction Dataset. <https://www.kaggle.com/datasets/patelris/crop-yield-prediction-dataset>
- scikit-learn — RandomForestRegressor. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- scikit-learn — LabelEncoder. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>
- scikit-learn — Regression Metrics. https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics
- Joblib Documentation — Persistence. <https://joblib.readthedocs.io/en/stable/persistence.html>
- FAO — World Food and Agriculture Statistical Yearbook. <https://www.fao.org/publications/card/en/c/CB4477EN>
- matplotlib — FigureCanvasTkAgg. https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html
- Python Tkinter ttk.Combobox. <https://docs.python.org/3/library/tkinter.ttk.html#combobox>
- Towards Data Science — Random Forest for Regression. <https://towardsdatascience.com/random-forest-regression-5f605132d19d>

9. Open-Meteo Weather API (Future Scope). <https://open-meteo.com/>
10. Real Python — Python Tkinter GUI Tutorial. <https://realpython.com/python-gui-tkinter/>